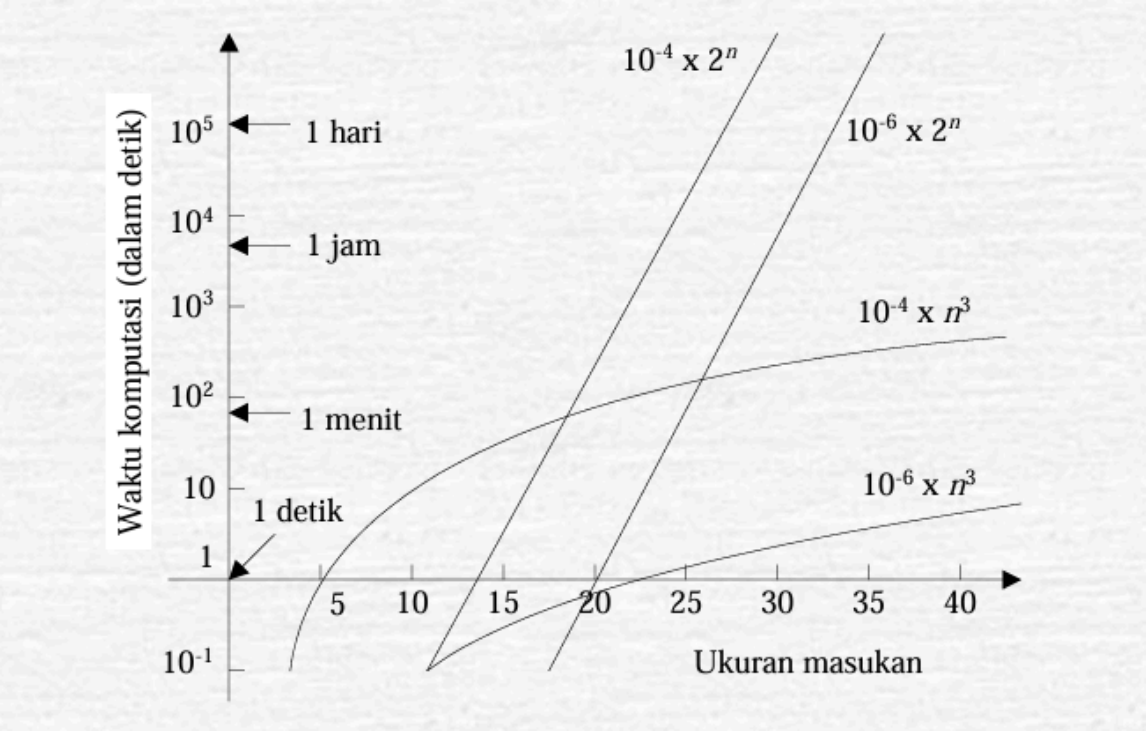
Kompleksitas Algoritma

1. Pendahuluan

Setiap permasalahan memiliki banyak algoritma penyelesaian. Misalkan: algoritma pengurutan (*sorting)* yang mempunyai puluhan algoritma. Sebuah algoritma tidak hanya harus benar tetapi juga mangkus (Efektif). Kemangkusan suatu algoritma dapat diukur melalui waktu (*time)* eksekusi serta kebutuhan ruang (*space)* memori. Suatu algoritma dikatakan efektif jika meminimumkan kebutuhan ruang dan waktu. Kebutuhan ruang dan waktu suatu algoritma bergantung pada ukuran masukan (*n)* yang menyatakan jumlah data yang diproses. Kemangkusan algoritma dapat digunakan untuk menilai algoritma yang bagus dari sejumlah algoritma penyelesaian. Perhatikan gambar di bawah ini.



Gambar di atas menunjukkan sebuah grafik yang membandingkan efisiensi algoritma. Sumbu X menyatakan ukuran masukan (n) dan sumbu Y menyatakan

waktu komputasi. Terdapat 4 orde kompleksitas waktu yang berbeda, yakni: $10^{-4} \times 2^n$, $10^{-6} \times 2^n$, $10^{-4} \times n^3$, $10^{-6} \times n^3$. Pertumbuhan algoritma eksponen (2^n) tampak lebih curam dibandingkan pertumbuhan algoritma polinom (n^3). Ini dapat dilihat pada ukuran masukan 20-30, di mana algoritma eksponen membutuhkan waktu hingga 1 jam sementara algoritma polinom kurang dari 1 jam. Dari grafik ini dapat disimpulkan bahwa algoritma polinom cenderung lebih efisien dibandingkan eksponen.

2. Model perhitungan kebutuhan waktu

Menghitung kebutuhan waktu algoritma dengan mengukur waktu sesungguhnya (dalam satuan detik) ketika algoritma dieksekusi oleh komputer bukanlah cara yang tepat. Hal ini dikarenakan: (1). Waktu setiap operasi antar satu komputer dengan komputer lain berbeda; (2). Waktu setiap operasi antar satu *compiler* dengan *compiler lain* tidak sama. Model abstrak pengukuran waktu/ruang haruslah independen dari pertimbangan mesin dan *compiler* manapun. Besaran yang digunakan untuk menerangkan model abstrak pengukuran waktu/ruang ini adalah kompleksitas algoritma. Terdapat dua macam kompleksitas algoritma, yaitu kompleksitas waktu dan kompleksitas ruang. Kompleksitas waktu [$T(n)$], diukur dari jumlah tahapan komputasi yang dibutuhkan untuk menjalankan algoritma sebagai fungsi dari ukuran masukan n. Kompleksitas ruang [$S(n)$] diukur dari memori yang digunakan oleh struktur data yang terdapat di dalam algoritma sebagai fungsi dari ukuran masukan n. Dengan menggunakan besaran kompleksitas waktu/ruang algoritma, kita dapat menentukan laju peningkatan waktu (ruang) yang diperlukan algoritma dengan meningkatnya ukuran masukan n. Dalam hal ini ukuran masukan n merupakan jumlah data yang diproses oleh sebuah algoritma. Contoh: algoritma pengurutan 1000 elemen larik (*array*), maka n = 1000.

3. Kompleksitas waktu

Jumlah tahapan komputasi dihitung dari berapa kali suatu operasi dilaksanakan di dalam sebuah algoritma sebagai fungsi ukuran masukan (*n*). Di dalam sebuah algoritma terdapat berbagai macam operasi: operasi baca/tulis, operasi aritmetika, operasi pengisian nilai (*assignment)*, operasi pengaksesan elemen larik, operasi pemanggilan fungsi/prosedur, dll. Contoh meninjau algoritma untuk mencari elemen terbesar dalam sebuah larik.

```
max ← A[0]
for i ← 1 to n-1 do
        if A[i] > max then
                max ← A[i]
        end if
end for
return max
```

Dari contoh di atas, operasi yang mendasar pada algoritma tersebut adalah A[i] > max, karena akan selalu dieksekusi pada setiap operasi loop dan menjadi penentu apakah variabel *max* perlu untuk diperbaharui atau tidak.

Kompleksitas waktu dibedakan atas tiga macam yakni:

1. $T_{max}(n)$ merupakan kompleksitas waktu terburuk (*worst case)* karena kebutuhan waktunya maksimum
2. $T_{min}(n)$ merupakan kompleksitas waktu terbaik (*best case)* karena kebutuhan waktunya minimum
3. $T_{avg}(n)$ merupakan kompleksitas waktu untuk kasus rata-rata (*average case).*

Kembali pada contoh di atas, mari kita hitung kompleksitas waktu serta mencari kasus terbaik dan kasus terburuknya.

```
procedure FindMax(A : array of n elements)
max ← A[0]
for i ← 1 to n − 1 do
   if A[i] > max then
      max ← A[i]
   end if
end for
return max
end procedure
```

Pseudocode	Cost	T_{min}	T_{max}
procedure FindMax	C1	1	1
max ← A[0]	C2	1	1
for i ← 1 to n − 1	C3	n	n
if A[i] > max	C4	n-1	n-1
max ← A[i]	C5	0	n-1
end if	C6	n-1	n-1
end for	C7	n	n
return max	C8	1	1
end procedure	C9	1	1
Total waktu (T_n)		$O(_n)$	$O(_n)$

Penjelasan:

Baris 1 dieksekusi 1 kali = C1 x 1 = C1

Baris 2 dieksekusi 1 kali = C2 x 1 = C2

Baris 3 dieksekusi n kali = C3 x n

Baris 4 dieksekusi n-1 kali = C4 (n-1)

Baris 5: best case: => kondisi selalu salah, artinya tidak pernah dieksekusi => C5 x 0 = 0

Worst case => kondisi selalu benar, artinya dieksekusi sebanyak n-1 kali => C5 (n-1)

Baris 6 dieksekusi n-1 kali = C6 (n-1)

Baris 7 dieksekusi n kali = C7 x n

Baris 8 dieksekusi 1 kali = C8 x 1 = C8

Baris 9 dieksekusi 1 kali = C9 x 1 = C9

Maka ditotalkan:

Best case: C1 + C2 + C3n + C4(n-1) + 0 + C6(n-1) + C7n + C8+ C9

C4 (n-1) => C4n-C4; C6 (n-1) => C6n-C6

Maka T_n = C1 + C2 + C3n + C4n-C4 + 0 + C6n-C6 + C7n + C8+ C9

= (C3+C4+C6+C7)n + (C1+C2+C8+C9−C4−C6)

= (1+1+1+1)n + (1+1+1+1-1-1)

= 4n+4 yang mana membentuk fungsi linear. karena fungsi linear maka Big-*O* => *O(n)*

Worst case: C1 + C2 + C3n + C4(n-1) + C5(n-1) + C6(n-1) + C7n + C8+ C9

C5(n-1) => C5n-C5

Maka T_n = C1 + C2 + C3n + C4n-C4 + C5n-C5 + C6n-C6 + C7n + C8+ C9

= (C3+C4+C6+C5+C7)n + (C1+C2+C8+C9−C4−C6-C5)

= (1+1+1+1+1)n + (1+1+1+1-1-1-1)

= 5n+3, sama-sama membentuk fungsi linear, maka Big-O => O(n)

4. Teorema-teorema

Teorema: Bila $T(n) = a_m n^m + a_{m-1} n^{m-1} + \ldots + a_1 n + a_0$ adalah polinom derajat m maka $T(n) = O(n^m)$. Jadi cukup melihat suku (*term)* yang mempunyai pangkat terbesar.

Contoh:

$T(n) = 5 = 5n^0 = O(n^0) = O(1)$

$T(n) = n(n - 1)/2 = n^2/2 - n/2 = O(n^2)$

$T(n) = 3n^3 + 2n^2 + 10 = O(n^3)$

Teorema tersebut digeneralisasi untuk suku dominan lainnya:

1. Eksponensial mendominasi sembarang perpangkatan (yaitu, $y^n > n^p$, $y > 1$)
2. Perpangkatan mendominasi ln n (yaitu $n^p > \ln n$)
3. Semua logaritma tumbuh pada laju yang sama (yaitu $a \log(n) = b \log(n)$)
4. $n \log n$ tumbuh lebih cepat daripada n tetapi lebih lambat daripada n^2

Contoh:

$T(n) = 2n + 2n^2 = O(2^n)$.

$T(n) = 2n \log(n) + 3n = O(n \log(n))$

$T(n) = \log(n^3) = 3 \log(n) = O(\log(n))$

$T(n) = 2n \log(n) + 3n^2 = O(n^2)$

Pengelompokkan algoritma berdasarkan Notasi *O-besar*

Kelompok Algoritma	Nama
O(1)	Konstan
O(log n)	Logaritmatik
O(n)	Lanjar
O(n log n)	n log n
$O(n^2)$	Kuadratik
$O(n^3)$	Kubik
$O(2^n)$	Eksponensial
O(n!)	Faktorial

Urutan spektrum kompleksitas waktu algoritma adalah:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < \ldots < O(2^n) < O(n!)$$

Penjelasan masing-masing kelompok algoritma sebagai berikut

Kompleksitas O(1) berarti waktu pelaksanaan algoritma adalah tetap, tidak bergantung pada ukuran masukan. Contohnya algoritma penukaran. Di sini, jumlah operasi penugasan ada sebanyak 3 buah (temp =a; a=b, b=temp). Jadi $T_n = 3 \Rightarrow O(1)$

Kompleksitas waktu logaritmik berarti laju pertumbuhan waktunya berjalan lebih lambat daripada pertumbuhan n. Algoritma yang termasuk kelompok ini adalah algoritma yang memecahkan persoalan besar dengan mentransformasikannya menjadi beberapa persoalan yang lebih kecil yang berukuran sama (misalnya

algoritma pencarian_biner). Di sini basis algoritma tidak terlalu penting sebab bila n dinaikkan dua kali semula, misalnya, log n meningkat sebesar sejumlah tetapan.

Algoritma yang waktu pelaksanaannya lanjar umumnya terdapat pada kasus yang setiap elemen masukannya dikenai proses yang sama, misalnya algoritma pencarian_beruntun. Bila n dijadikan dua kali semula, maka waktu pelaksanaan algoritma juga dua kali semula.

Waktu pelaksanaan yang n log n terdapat pada algoritma yang memecahkan persoalan menjadi beberapa persoalan yang lebih kecil, menyelesaikan tiap persoalan secara independen, dan menggabung solusi masingmasing persoalan. Algoritma yang diselesaikan dengan teknik bagi dan gabung mempunyai kompleksitas asimptotik jenis ini. Bila n = 1000, maka n log n mungkin 20.000. Bila n dijadikan dua kali semual, maka n log n menjadi dua kali semula (tetapi tidak terlalu banyak)

Algoritma yang waktu pelaksanaannya kuadratik hanya praktis digunakan untuk persoalana yang berukuran kecil. Umumnya algoritma yang termasuk kelompok ini memproses setiap masukan dalam dua buah kalang bersarang, misalnya pada algoritma urut_maks. Bila n = 1000, maka waktu pelaksanaan algoritma adalah 1.000.000. Bila n dinaikkan menjadi dua kali semula, maka waktu pelaksanaan algoritma meningkat menjadi empat kali semula.

Seperti halnya algoritma kuadratik, algoritma kubik memproses setiap masukan dalam tiga buah kalang bersarang, misalnya algoritma perkalian matriks. Bila n = 100, maka waktu pelaksanaan algoritma adalah 1.000.000. Bila n dinaikkan menjadi dua kali semula, waktu pelaksanan algoritma meningkat menjadi delapan kali semula.

Algoritma yang tergolong kelompok ini mencari solusi persoalan secara "brute force", misalnya pada algoritma mencari sirkuit Hamilton Bila n = 20, waktu

pelaksanaan algoritma adalah 1.000.000. Bila n dijadikan dua kali semula, waktu pelaksanaan menjadi kuadrat kali semula!

Seperti halnya pada algoritma eksponensial, algoritma jenis memproses setiap masukan dan menghubungkannya dengan n - 1 masukan lainnya, misalnya algoritma Persoalan Pedagang Keliling Bila n = 5, maka waktu pelaksanaan algoritma adalah 120. Bila n dijadikan dua kali semula, maka waktu pelaksanaan algoritma menjadi faktorial dari 2n.

Studi kasus

Sebuah sistem akademik menyimpan n data mahasiswa dalam sebuah larik (array). Sistem tersebut sering digunakan untuk mencari satu mahasiswa berdasarkan NIM. Terdapat dua algoritma yang dipertimbangkan:

Pencarian Beruntun (Sequential Search)

Pencarian Biner (Binary Search)

Tinjau dari kedua algoritma tersebut untuk menentukan algoritma yang paling efisien berdasarkan kompleksitas waktu.